RAG Assignment: Baseline vs. Re-Ranking

Submission Deadline: 19th October 2026 **Deliverables:** GitHub Link + Report

Important Submission Instructions

- **Code Submission:** All code must be uploaded to a public or shared GitHub repository and submitted. It is strongly preferred to have **a single Jupyter Notebook (`.ipynb`)** containing well-commented code explaining each step of your work.
- **Report Authenticity:** The report should **not** be AI-generated. Keep answers directly relevant to what was asked, concise, and in your own words. We want to evaluate your actual understanding of the concepts and observations.

1. Setup & Environment

- **LLM & Embeddings:** You may use the Gemini API, any alternative LLM API (OpenAI, Anthropic), or local open-weights models (via Ollama, Hugging Face, etc.).
- **Dataset:** Select a single, complex PDF document. Suitable options include a technical policy, regulatory/legal document, or an in-depth board game rulebook.

2. Two-Stage Retrieval Implementation

You will implement and contrast two retrieval workflows over the exact same chunked document corpus:

- **Pipeline 1 (Baseline Retrieval):**
  a. Index chunks into a vector store using a dense embedding model (Bi-Encoder).
  b. For a given query, retrieve the **Top 10** chunks based on cosine similarity or inner product.
  c. Take the **Top 3** chunks directly and provide them to your LLM prompt to generate **Answer 1**.
- **Pipeline 2 (Advanced Re-Ranking):**
  a. Take the **Top 10** candidates returned from the initial baseline retrieval.
  b. Pass all 10 (query, chunk) pairs through a Cross-Encoder model (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2`).
  c. Re-sort candidates descending by Cross-Encoder score and select the **Top 3** re-ranked chunks.
  d. Provide these re-ranked chunks to the LLM (using the identical prompt structure) to generate **Answer 2**.

3. Failure Mode Analysis (Stress Testing)

To highlight the practical benefit of a Cross-Encoder, evaluate **at least two specific failure queries** where the baseline retrieval stumbles or degrades, but re-ranking restores relevance.

Along with the above cases, you must also demonstrate document-specific failures you uncover, such as:

- **Keyword Trap / Distractor:** A query with high lexical overlap to an irrelevant passage, while the true answer uses distinct terminology or synonyms.

- **Exceptions & Corner Cases:** The baseline retrieves a general rule chunk, but misses a specific restriction, penalty, or "except when…" condition located elsewhere.
- **Negation & Constraints:** Queries with qualifiers such as "not allowed", "prohibited", or specific role limits where Bi-Encoder embeddings blur negative and positive context.
- **Other Document-Specific Failures:** Any scenario where a vital chunk is buried at ranks 4–10 initially, but gets successfully promoted into the Top 3 by the Cross-Encoder.

4. Analysis and Report

Submit a concise report addressing the following four areas:

1. **Architecture Comparison:** Briefly explain the core difference between a Bi-Encoder (independent embeddings, cosine similarity) and a Cross-Encoder (joint self-attention across tokens).
2. **Qualitative & Rank Shift Analysis:**
   - Document the rank changes for your failure queries (e.g., Chunk X moved from Rank #8 $\rightarrow$ Rank #1).
   - Compare LLM Answer 1 vs. Answer 2: Did the baseline generate a hallucination, omit an essential clause, or misstate a rule because the critical chunk sat outside the top 3?
3. **Latency Profiling:** Record and tabulate the runtime of the baseline retrieval vs. the re-ranking pipeline.
4. **Scaling Question:** Explain why we do not simply run a Cross-Encoder across the entire document collection from the very start.